Field Hours

CSCE 626 Parallel Algorithm Design and Analysis

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Work, span, and Brent

You charge a parallel algorithm with two numbers. Work W is every operation a single processor would perform. Span S is the longest chain of dependent operations. Summing 8 numbers in a balanced tree uses W = 7 additions, because each addition removes one partial sum, and the span is S = 3, one level each time you halve 8 down to 1. You cannot report a parallel time from W alone.

You turn work and span into a time on P processors with Brent's bound: T is at most W/P + S. On the sum of 8 numbers with P = 4, that is 7/4 + 3 = 4.75. A careful schedule stays inside the bound, and a careless assignment of additions to processors can idle everyone and do worse. The bound is an upper bound on a good schedule. It is not a promise about every schedule.

Adding eight ones

8 leaves at the start

4 sums on level 1

2 sums on level 2

1 total, span 3

Sum of eight ones

1. Pair 8 ones into four 2s, then two 4s, then a single 8.

2. The additions are 4 + 2 + 1 = 7, so W = 7. The longest chain has S = 3 levels.

3. With P = 4, Brent's bound is 7/4 + 3 = 4.75. The tree itself finishes in 3 levels if 4 processors are free at the first level.

Problem 1. Sum 16 numbers in a balanced tree. What are W and S?

Answer. W = 15 and S = 4. You need 15 additions to leave one total, and log base 2 of 16 is 4.

Problem 2. On the sum of 8 numbers, what is Brent's bound with P = 2?

Answer. 7/2 + 3 = 6.5. Fewer processors raise the bound from 4.75 to 6.5.

Lecture 2 · 90 minutes

Speedup when the chain is short

You define speedup as sequential time divided by parallel time. If the sequential time is the work W, and no schedule beats the span S, then speedup is at most W/S. On the sum of 8 numbers, W/S = 7/3, about 2.33. A

machine with 100 processors still cannot finish in fewer than 3 dependent steps. Most of those processors have no addition to run on the last level, where only 1 addition remains.

You get efficiency by dividing speedup by P. The tree on 8 numbers finishes in 3 steps on P = 4, so speedup is 7/3 and efficiency is (7/3)/4 = 7/12. The first level has 4 additions and the last has 1, so those 4 processors cannot stay full. You match P to the width of the dag rather than to the machine you happen to own.

Same span, more processors

P = 4

Finish in 3 steps

Efficiency is 7/12

P = 100

Span still blocks you

Speedup at most 7/3

Ceiling on the sum tree

1. Work is 7 and span is 3, so the speedup ceiling is 7/3.

2. On 4 processors, finishing in 3 steps gives efficiency (7/3) divided by 4, which is 7/12.

3. On 100 processors the span is still 3, so speedup cannot exceed 7/3. Extra processors do not shorten the chain.

Problem 1. A dag has W = 100 and S = 5. What is the largest possible speedup if you buy P = 40?

Answer. Speedup is at most 100/5 = 20. Forty processors cannot beat span 5, so the ceiling stays 20, not 40.

Problem 2. The sum tree finishes in 3 steps on P = 4 with W = 7. What is the efficiency as a fraction?

Answer. Speedup is 7/3. Efficiency is (7/3)/4 = 7/12.

Lecture 3 · 90 minutes

Prefix sums in two sweeps

You compute a prefix sum by replacing x1 through xn with the sums of the growing prefixes. A work-efficient scan uses two sweeps. The up-sweep is a reduction tree. The down-sweep fills each slot from a sibling partial sum. For n = 8 each sweep has depth 3, so the span of both is 6, while the work stays under about 2n additions. A sequential scan on the same array has a chain of length 8.

You check n = 4 on the input 1, 2, 3, 4. The up-sweep writes 1, 3, 3, 7 and then 1, 3, 3, 10. Clear the total to 0 and the down-sweep yields exclusive prefixes 0, 1, 3, 6. Add the original entries to get inclusive prefixes 1, 3, 6, 10. Two sweeps have span 2 times log base 2 of 4, which is 4.

Up-sweep, then down-sweep

Up-sweep to the total -> Clear the total to 0 -> Down-sweep the prefixes -> Add the original entries

Scan of 1, 2, 3, 4

1. Up-sweep: the array becomes 1, 3, 3, 7 and then 1, 3, 3, 10.

2. Set the last entry to 0 and down-sweep. The exclusive prefixes are 0, 1, 3, 6.

3. Add the originals: 0+1, 1+2, 3+3, 6+4 gives the inclusive scan 1, 3, 6, 10.

Problem 1. What is the inclusive prefix scan of 2, 2, 2, 2?

Answer. 2, 4, 6, 8. Each entry is the sum of the inputs up through that index.

Problem 2. For n = 8, the work-efficient scan has span 2 times log base 2 of 8. What is that span, and why is it not 8?

Answer. Log base 2 of 8 is 3, so the span is 6. A sequential scan has a chain of length 8. The two sweeps cut the chain to 6 on this size.

Lecture 4 · 90 minutes

Pointer jumping on a list

You rank a linked list by pointer jumping, since a tree scan needs random access. Each round sends a node to its successor's successor, so round r reaches 2 to the r hops ahead. A list of 8 nodes finishes in 3 rounds, because 2 cubed is 8. Each node starts at rank 1 and adds the rank it jumps over. One round reads the old ranks, then writes.

You can finish a list of 4 nodes in 2 rounds. Ranks start at 1, 1, 1, 1. After round 1 they are 2, 2, 2, 1. After round 2 they are 4, 3, 2, 1, and the head holds the length 4. On a list of 8, one processor per node does 8 times 3 = 24 updates, and the span is 3. The original chain is now a log-depth circuit.

Ranks on four nodes

1. Start at rank 1 each

2. Round 1 writes 2, 2, 2, 1

3. Round 2 writes 4, 3, 2, 1

4. Head rank equals length 4

Ranking A, B, C, D

1. Nodes A, B, C, D start with ranks 1, 1, 1, 1.

2. After one synchronous round the ranks are 2, 2, 2, 1.

3. After the second round the ranks are 4, 3, 2, 1. The head holds 4, and the span was 2 rounds.

Problem 1. A list has 16 nodes, each starting at rank 1. How many rounds does the head need, and what rank does it hold at the end?

Answer. 4 rounds, because 2 to the 4th is 16. The head's rank is 16, the number of nodes from the head through the tail.

Problem 2. On the 4-node list, why is the second node's rank 3 rather than 4?

Answer. Three nodes remain from that node through the tail. Round 1 gave it rank 2, and round 2 added the tail's rank 1, totaling 3.

Lecture 5 · 90 minutes

Area against time on a cut

You charge area when the schedule is laid out as a chip. A cut of w wires that must move n bits needs time T at least n/w, because one step moves at most w bits. If the cut length is w, this counting model takes area A at least w squared, and width 2 means area 4. Then A times T squared is at least n squared. You can trade area for time. You do not delete the product.

You set n = 8 and w = 2. Time is at least 8/2 = 4, and area is at least 4. Then A T squared is at least 4 times 16 = 64, matching n squared. At w = 4, time falls to at least 2 and area rises to at least 16, and 16 times 4 is still 64. You buy time with area. In this model the product does not drop.

Two cuts, same product

Width w = 2

Time at least 4

Area at least 4

Width w = 4

Time at least 2

Area at least 16

Eight bits through two wires

1. With w = 2, time is at least 8/2 = 4 steps.

2. Area is at least w squared = 4, so A T squared is at least 4 times 16.

3. That product is 64, equal to n squared. Widening to w = 4 still yields 16 times 4 = 64.

Problem 1. For n = 8 you want T at most 2. What wire count does T at least n/w require, and what area follows?

Answer. w at least 8/2 = 4. Area at least 16. Then A T squared is at least 16 times 4 = 64.

Problem 2. Why do you reject a proposed layout with A = 4 and T = 2 for n = 8?

Answer. A T squared = 4 times 4 = 16, which is below n squared = 64. This counting model forbids it.

Lecture 6 · 90 minutes

Fan-in lower bounds

You lower-bound span by fan-in before you draw a chip. A gate with 2 inputs forms a tree, so an output that depends on n bits needs depth at least log base 2 of n. Parity of 8 bits depends on every bit: flip one input and the output flips. The span is at least 3. Depth 2 sees at most 4 inputs, so it cannot compute parity on every 8-bit

string.

You see the same bound on a 3 by 3 window. Nine neighbors cannot feed one gate of fan-in 2. A binary tree on 9 leaves has depth 4, because depth 3 holds at most 8 leaves. One processor per pixel does not cut that depth. More pixels add work across the image. They do not shorten the chain inside the window.

Parity tree on eight bits

8 input bits

4 gates on level 1

2 gates on level 2

1 parity bit, depth 3

Parity of eight bits

1. The bits 1, 0, 1, 1, 0, 0, 1, 0 contain four 1s, so the parity is 0.

2. A binary tree computes that bit in 3 levels: 4 gates, then 2, then 1.

3. A circuit of depth 2 sees at most 4 inputs. It cannot be correct for every 8-bit string.

Problem 1. Parity of 32 bits, gates of fan-in 2. What is the minimum depth?

Answer. Log base 2 of 32 is 5. A shallower circuit cannot depend on all 32 inputs.

Problem 2. Why is the depth of a binary sum of 9 window pixels at least 4, not 3?

Answer. Depth 3 has at most 8 leaves. The ninth pixel needs another level. Depth 4 is the minimum.

Lecture 7 · 90 minutes

A short filter, scheduled

You schedule a width-3 filter as short chains. Each output adds a sample to its two neighbors. On n = 8 padded samples, each output does 2 additions, so the work is 16 and the span is 2. The 8 outputs are independent. Brent with P = 8 gives T at most 16/8 + 2 = 4. The short span is the local dependency, not the name of the task.

You set that filter next to parity of the same 8 bits. Parity had work 7 and span at least 3, because the bit depends on all 8 inputs. The filter has more work, 16, and a shorter span, 2. A window of width 8 would force span at least 3 again. You read the bound from the dependency, not from the application name.

One output, three taps

Read three samples -> Add the first pair -> Add the third sample -> Write that output

Third sample of 1 through 8

1. The window on the sample 3 covers 2, 3, and 4.

2. Add 2 + 3 = 5, then 5 + 4 = 9. That output has span 2.

3. Eight such outputs give work 16. With P = 8, Brent's bound is 16/8 + 2 = 4.

Problem 1. Same filter, n = 8, but P = 4. What is Brent's bound?

Answer. 16/4 + 2 = 6.

Problem 2. You widen the window from 3 taps to 8, so each output depends on all 8 samples. What replaces the span of 2?

Answer. A binary tree on 8 inputs has depth at least 3. The span is at least 3. The old span of 2 used the narrow window.

Lecture 8 · 90 minutes

What you can now bound

You can now state work, span, and a Brent bound, rank a list by pointer jumping, and scan an array in two sweeps. You can also tell when a chip cut forbids a schedule the graph allowed. Summing 8 numbers has W = 7 and S = 3, so P = 4 yields 7/4 + 3 = 4.75. Moving 8 bits through w = 2 wires needs time at least 8/2 = 4.

You hold the two limits apart. The span says the additions cannot finish before step 3. The cut says the wires cannot finish before step 4 if only 2 wires cross. A design with T = 3 can satisfy the tree and still be impossible as that layout. Parity of 8 bits also needs depth 3, the same number of rounds pointer jumping used on a list of 8. You write both bounds down before you claim a speedup.

Span versus the cut

Addition tree

Span S = 3

Brent bound is 4.75

Cut of width 2

Time at least 4

Product at least 64

Three is not always enough

1. Summing 8 numbers has W = 7 and S = 3. Brent on P = 4 allows up to 4.75.

2. Moving 8 bits on w = 2 wires needs T at least 4, with A T squared at least 64.

3. Time 3 meets the span and misses the cut. You need time at least 4, or a wider cut.

Problem 1. For the sum of 8 numbers on P = 4, is a running time of 4 allowed by both Brent and the w = 2 cut?

Answer. Yes. Brent allows up to 4.75, and the cut requires at least 4. Time 4 meets both. Time 3 meets the span and fails the cut.

Problem 2. Combine pointer jumping with fan-in. A list of 8 nodes is ranked in 3 rounds. Parity of those 8 weights needs depth at least 3. Can you compute that parity in depth 2 by skipping the ranking?

Answer. No. Fan-in 2 forces depth at least 3 on 8 bits, whether or not you also compute ranks.

Ranking in 3 rounds matches that depth and returns every rank, not only the parity bit.